MAEG4060 Virtual Reality Systems and Applications

The Chinese University of Hong Kong (CUHK)
Course Project – Final Stage Report

Project title: Interactive Physical Simulation of Off-Road Vehicle Dynamics in a Mountainous Terrain

Group members:

Guo Tsun Hei (1155186272): 3D Modeling, Texturing, Rigging, and Animation
Yim Fu Chong (1155176974): Programming, Physics Engine, and System Integration

Submission deadline: 17 April 2026

Abstract

We present a DirectX 9.0c fixed-function **sandbox** (not a scored game) set on procedural mountainous terrain. The user drives a textured vehicle with keyboard or XInput gamepad; a third-person camera follows the chassis. The terrain uses bilinear height sampling, slope-based splatting (grass base plus blended rock, stone, and mountain layers), and surface normals for vehicle orientation. Static props include a crate (AABB) and a loose boulder (bounding sphere with gravity and impulses). A windmill model is loaded through the DirectX frame hierarchy with `ID3DXAnimationController` when present in the asset. A simple day/night cycle modulates the sun direction, ambient light, and clear colour; an `ID3DXFont` HUD shows speed and position. The implementation is native Visual C++17 with CMake, satisfying the course requirements for real-time 3D, input, physics-based motion, navigation on geometry, collision, and Maya-exported animation integration.

Contents

1 Introduction

1.1 Project scope

The environment is an off-road hillside: the player explores a heightfield, interacts with props, and observes a time-varying sky palette. Compared to the Stage 1 proposal, terrain height data are **generated procedurally** at startup (Sketchfab terrain remained a visual reference) to keep the repository self-contained; splatting, vehicle dynamics, and collision behaviour match the intended design.

1.2 Alignment with course requirements

The application is implemented in **Visual C++** with **DirectX 9.0c**. It is a technical demonstration / sandbox (**not** a video game: no missions, no score). Input: **keyboard** and **gamepad**. The scene includes **physical simulation** (e.g. gravity, vehicle dynamics, terrain interaction, collisions).

1.3 Mapping to marking criteria (group project)

Table 1 maps Stage 3 marking components from the course project description to sections in this report.

Table 1: Marking components and where they are described.

Component (weight)	**See section**	**Notes**
Gamepad / joystick (5%)	§3.3	XInput / analog mapping
Model navigation (5%)	§4.3	Terrain following, camera or vehicle
Collision, static objects (10%)	§4.5	e.g. AABB / spheres
Physics-based motion (10%)	§4.6	Forces, integration, friction
Maya animation in VR (20%, group)	§4.4	`.X` / `ID3DXAnimationController` etc.
User guide (5%)	§3	Controls, requirements, run instructions
Report (15%)	Whole document	Clarity, completeness
Bonus (10%)	§4.8	Moving-object collision, extra effects, ...

2 Completed application

2.1 Feature overview

Procedural terrain mesh with bilinear sampling; grass textured base mesh then alpha-blended splats for rock, stone, and mountain where slope/height masks apply (`src/Terrain.cpp`); simple longitudinal vehicle dynamics with steering, rolling resistance, and braking; kinematic terrain following (pose from surface normal); chase camera; crate and boulder with AABB/sphere collision; boulder under gravity with impulse from the vehicle hull; hierarchical windmill draw with animation time advanced each frame; day/night lighting and clear colour; on-screen HUD (`ID3DXFont`); resize-safe device reset path for the font.

2.2 Executable and source layout

Submit the Release binary as `Game_Build/maeg4060_stage2.exe` together with `Game_Build/Assets/` (see `scripts/package_game_build.bat` or CMake `install`). Source layout:

- Entry: `src/main.cpp`; simulation/render orchestration: `src/Game.cpp`.

- Rendering/device: `src/Graphics.cpp`; camera: `src/Camera.cpp`; static meshes: `src/ModelLoader.cpp`; hierarchy/animation: `src/AnimatedModel.cpp`; terrain: `src/Terrain.cpp`; vehicle: `src/Vehicle.cpp`; collision helpers: `src/Collision.cpp`; input: `src/Input.cpp`.
- Build: `CMakeLists.txt` (target `maeg4060_stage2`, links `d3d9`, `d3dx9`, `xinput9_1_0`).
- Assets: static props now support `Assets/models/replacements/*.obj + *.mtl + textures` with fallback to legacy `Assets/models/*.x`; terrain textures are generated at runtime if no image files are supplied.

2.3 Screenshots

Figures 1 and 2 show the running DirectX 9.0c sandbox and a GitHub Actions run page (workflow #14: boulder and windmill models) displaying the same application.

3 User guide

3.1 System requirements

Windows 10 or later (64-bit recommended), DirectX 9 compatible GPU, display at least 1280 × 720. D3DX is supplied at build time either via the June 2010 DirectX SDK (`DXSDK_DIR`) or CMake-downloaded `Microsoft.DXSDK.D3DX` (see Appendix A); the packaged build may include `D3DX9_43.dll`. Optional Xbox-compatible controller (XInput).

3.2 Installation and running

Run `maeg4060_stage2.exe` from `Game_Build` with `Assets` alongside it, or run from the repo root after CMake sets the VS debugger working directory. Build instructions are in the standalone user guide PDF.

3.3 Controls

3.3.1 Keyboard

Input	Action
W / S	Throttle forward / reverse
A / D	Steer left / right
Space	Brake
M	Pause / resume windmill animation (when available)
Esc	Exit application

3.3.2 Gamepad

Left stick: steering. Right trigger: forward throttle. Left trigger: brake and partial reverse (see `src/Input.cpp`).

3.4 Troubleshooting

Missing `d3dx9_*.dll`: install the legacy DirectX end-user runtime. Missing models: verify `Assets/models` relative to the process working directory. Black screen after resize: update GPU drivers; the sample calls `ID3DXFont::OnLostDevice`/`OnResetDevice` around `IDirect3DDevice9::Reset`.

Figure 1: In-application view: procedural terrain with grass/rock/stone splats, windmill with hierarchy animation, props with crate texture, and HUD (speed, camera position, splat summary).

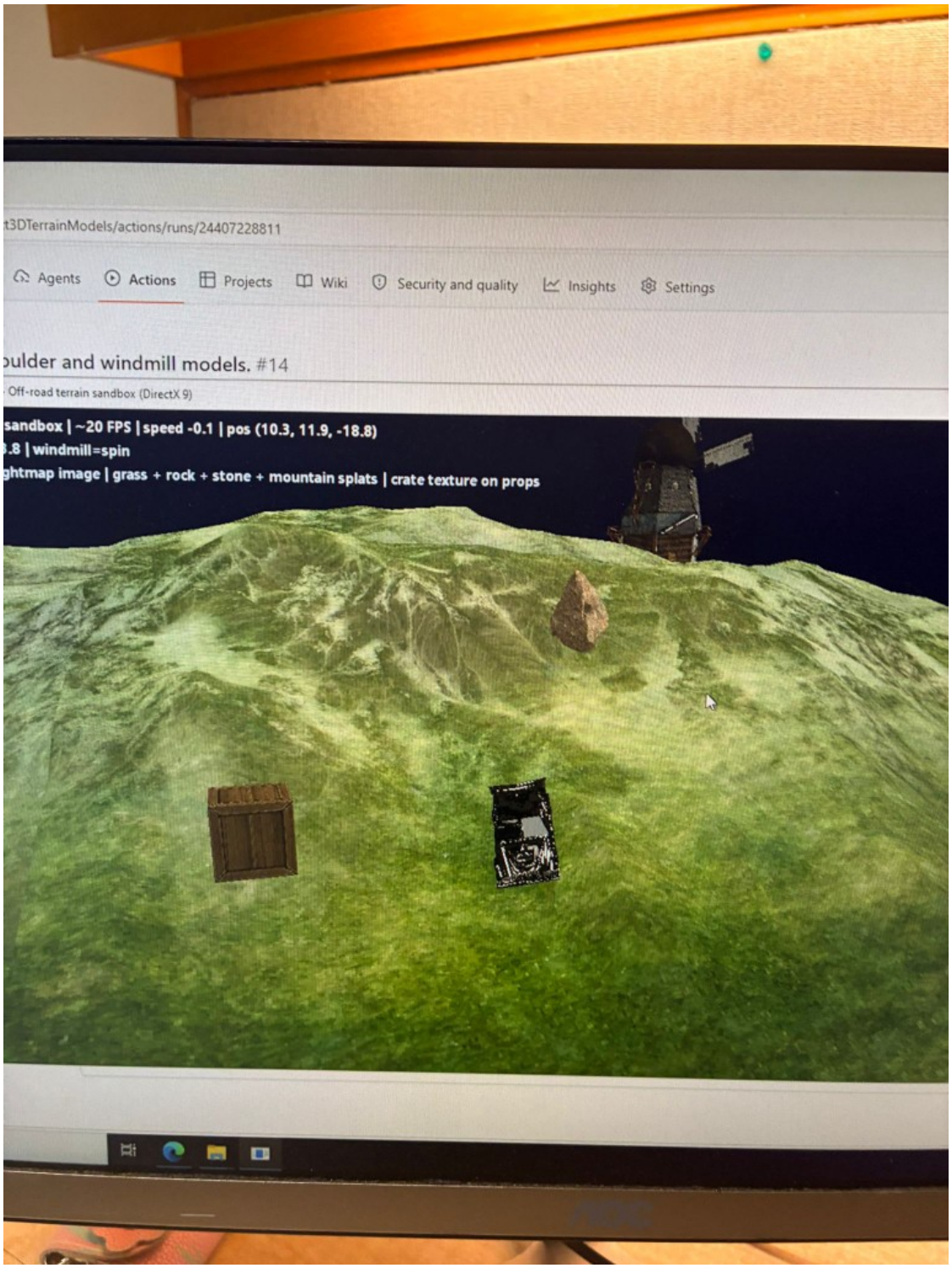

Figure 2: GitHub Actions: workflow run documenting boulder and windmill integration; inset shows the sandbox with windmill, boulder, crate, vehicle, and terrain HUD.

4 Technical description

This section summarizes the main techniques used in the DirectX 9.0c (fixed-function pipeline) implementation: terrain sampling, vehicle–terrain coupling, collision choices, animation, lighting, and environmental effects. The application is a physics sandbox (not a scored game).

4.1 Architecture overview

The real-time loop follows input → simulation → rendering: keyboard via `GetAsyncKeyState`, gamepad via `XInputGetState`; `Game::Update` integrates the vehicle and boulder, resolves collisions, advances animation time, and updates lighting parameters; `Game::Render` sets the chase camera, draws terrain, hierarchy model, props, and vehicle with the fixed-function pipeline and a font overlay.

4.2 DirectX 9 rendering

The renderer uses a Direct3D 9 device with a depth–stencil buffer, textured meshes, and directional lighting. Meshes and animations are loaded from the `.X` format; the coordinate system is left-handed, consistent with `D3DXMatrixLookAtLH` and standard DirectX samples.

4.3 Terrain and model navigation

4.3.1 Bilinear interpolation for terrain height

The terrain height at an arbitrary world position (x, z) is obtained from a heightmap grid. To avoid discrete "stair-stepping" when the vehicle moves, the engine samples the four neighbouring heightmap texels and applies **bilinear interpolation**:

$$h(x,z) = (1-t_x)(1-t_z)\,h_{00} + t_x(1-t_z)\,h_{10} + (1-t_x)t_z\,h_{01} + t_x t_z\,h_{11},$$

where $t_x, t_z \in [0,1]$ are fractional coordinates within the cell. This yields a continuous height field suitable for stable contact and visual alignment.

4.3.2 Surface normal alignment

The vehicle's up vector is aligned with the **terrain surface normal**, computed from the local geometry (e.g. cross product of edges on the heightfield triangle under the vehicle). Pitch and roll follow the normal while yaw is driven by steering input, giving kinematic terrain-following without penetrating the mesh.

4.4 Animation

Maya keyframed animation for the windmill is exported in the `.X` format. The runtime uses `D3DXLoadMeshHierarchyFromX` with an `ID3DXAllocateHierarchy` implementation (`src/AnimatedModel.cpp`) and advances `ID3DXAnimationController` each frame. The jeep mesh is drawn as a single subset from `D3DXLoadMeshFromX` without per-wheel hierarchy in this build.

4.5 Collision detection and response

4.5.1 Axis-aligned bounding boxes (AABB)

Rectangular props (e.g. wooden crates) use **AABB** tests: fast intersection in world space along x, y, and z extents. Response can slide or push the vehicle along the minimum penetration axis.

4.5.2 Bounding spheres

Irregular objects (e.g. boulders) use **bounding sphere** collision: distance between sphere centres compared to the sum of radii. Spheres are cheap to update when objects rotate or roll and pair well with impulse-based response for loose rocks.

4.5.3 Dynamic boulder interaction

Vehicle–boulder contact applies positional correction and a simple impulse so the boulder can roll on slopes; sphere tests against the vehicle AABB and crate AABB are used (details in §4.8).

4.6 Physics and simulation

Gravity is applied to the boulder as a constant downward acceleration (engine constant ≈ 28 units/s^2 for stable stepping). The vehicle uses semi-implicit longitudinal integration with throttle, rolling resistance, and brake terms (`src/Vehicle.cpp`).

4.7 Environmental effects

A **day/night cycle** orbits the directional light and interpolates clear and ambient colours (`src/Game.cpp`). **Texture splatting** draws a grass-textured heightfield, then alpha-blends rock (vertex diffuse alpha from slope), then optional stone and mountain overlays on dedicated vertex streams where mask weights are non-zero. The HUD shows speed, world position, simulation time, and windmill state.

4.8 Bonus work

Partial bonus: the boulder is dynamic (gravity, velocity, sphere versus vehicle AABB and crate AABB with positional correction and simple impulse when hit by the vehicle). Full rigid-body stacking and boulder–boulder coupling are not implemented.

5 3D models: sources and modifications

Per course requirements, external assets are listed with Sketchfab links and licenses. The **modifications** bullets summarize how each asset was adapted for this engine (scale, export format, collision, in-engine fallback paths).

1. **Half Dome Terrain Landscape**
   - Author: Studio Lab
   - License: Creative Commons Attribution (CC BY 4.0)
   - Source: Sketchfab
   - URL: `https://sketchfab.com/3d-models/half-dome-terrain-landscape-86ef075dc7d9414583ac2180e1e651a3`
   - Usage: Reference for high-resolution terrain geometry and heightmap generation.
   - Modifications: In-engine procedural heightfield (129×129 vertices, 200×200 world units) with analytic multi-frequency hills instead of the original Sketchfab mesh; runtime-generated grass/rock textures.
2. **Controllable Vehicle (ATV)**
   - Title: ATV from game “PURE”
   - Author: barking_dogo

- License: Creative Commons Attribution (CC BY 4.0)
- Source: Sketchfab
- URL: `https://sketchfab.com/3d-models/atv-from-game-pure-f2e454611ebd4151b9d7aa31815c35e4`
- Usage: Main controllable vehicle model (wheel nodes separated for procedural animation).
- Modifications: Shipped as Wavefront OBJ under `Assets/models/replacements/model_jeep/` when present, with `model_jeep.x` as fallback; scaled and placed with a world matrix from terrain normal and yaw; collision hull is a tuned AABB.

3. **Rock Boulder (Game Ready Asset)**

- Author: Aparicio Silva 3D
- License: Creative Commons Attribution (CC BY 4.0)
- Source: Sketchfab
- URL: `https://sketchfab.com/3d-models/rock-boulder-game-ready-asset-0079cb0e717c4aeebe6bca21ec21b0d9`
- Usage: Dynamic physics / bounding-sphere collision.
- Modifications: OBJ under `Assets/models/replacements/model_boulder/` when present, else `model_boulder.x`; uniform scale for gameplay radius ≈ 2.2; sphere collision and ground contact at terrain height plus radius.

4. **Windmill**

- Author: KIFIR
- License: Creative Commons Attribution (CC BY 4.0)
- Source: Sketchfab
- URL: `https://sketchfab.com/3d-models/windmill-fe13862b9e3c4dc887d10e297d86f17f`
- Usage: Keyframed animation and environmental aesthetics.
- Modifications: OBJ under `replacements/model_windmill/` when present, else `model_windmill.x`; Maya keyframes in `.x`; playback via `ID3DXAnimationController` when animation sets exist.

5. **Wooden Crate**

- Author: Domingos Studios
- License: Creative Commons Attribution (CC BY 4.0)
- Source: Sketchfab
- URL: `https://sketchfab.com/3d-models/wooden-crate-3c0259d17f3b4306890cdc809b545670`
- Usage: Static collision (AABB).
- Modifications: OBJ under `replacements/model_crate/` when present, else `model_crate.x`; world placement at $(-35, h, 48)$ with AABB half-extents $(2.2, 2.2, 2.2)$ for collision.

6 Conclusion

The sandbox meets the core Stage 3 requirements in a compact codebase. Known limitations: no audio, no skinned character pipeline, jeep wheels are not independently articulated in code, and

terrain is procedural rather than artist-authored DEM data. Future work could import a real heightmap PNG, add wheel bones from hierarchy, and extend physics to full rigid-body contact.

7 References

- Course project description (MAEG4060, 2025–26 Semester 2).
- DirectX SDK / MSDN documentation.
- External assets: cite consistently with §5.

A Appendix A: Build configuration

The project is a **Windows** CMake target `maeg4060_stage2` (`WIN32` subsystem), C++**17**, `cmake_minimum_required(3.20)`. MSVC is used with warning level `/W4` and static runtime selection `MultiThreaded` (Release) / `MultiThreadedDebug` (Debug). Preprocessor definitions `UNICODE` and `_UNICODE` are set on the main executable (and on the small helper targets `empty_window` and `d3d_device` in the same `CMakeLists.txt`).

D3DX headers and import library: either (1) set environment variable `DXSDK_DIR` to the root of the **June 2010 DirectX SDK** (folders `Include` and `Lib/x64` or `Lib/x86`), or (2) leave it unset so CMake downloads NuGet package `Microsoft.DXSDK.D3DX` version **9.29.952.8** into the build tree on first configure (requires network once), then uses its `build/native/include` and `lib` paths. If the NuGet path is used, a post-build step copies `D3DX9_43.dll` next to the executable when present.

Linked system / redistributable APIs: `d3d9`, `d3dx9`, `xinput9_1_0`. A post-build step copies the `Assets/` directory beside the built `.exe`. CMake `install` installs the game executable, `Assets/`, and the D3DX DLL when applicable (`scripts/package_game_build.bat` or `cmake -install` for a `Game_Build`-style layout).

Recommendation: Release configuration for markers; Ninja or Visual Studio generators; 64-bit (`x64`) builds match the documented D3DX library paths.

B Appendix B: Third-party libraries and runtime components

The application **does not** link separate general-purpose C++ libraries (no Eigen, Boost, or similar in `CMakeLists.txt`). Mathematics and mesh helpers use **D3DX** (`D3DXMath`, mesh loaders) from the same D3DX stack as Appendix A.

Microsoft components (linked or loaded at runtime):

- **Direct3D 9** (`d3d9.dll`) — system component on Windows; used for device creation, fixed-function pipeline drawing, and textures.
- **D3DX 9** (`d3dx9_*.dll` / import library `d3dx9.lib`) — legacy D3DX utilities (math, mesh, font, effect helpers). Headers/libs from June 2010 SDK or from the `Microsoft.DXSDK.D3DX` NuGet package; redistribution of `D3DX9_43.dll` follows Microsoft's terms for that package or the legacy DirectX end-user runtime, as applicable to your distribution channel.
- **XInput** (`xinput9_1_0.dll`) — gamepad input; typically present on Windows.

3D assets (Sketchfab models and textures) are listed with licenses in §5; they are data files under `Assets/`, not compile-time libraries.

C Appendix C: User Guide (standalone PDF)

The following pages reproduce the submitted user guide (`User_Guide.pdf`): controls (keyboard and gamepad), hardware and software requirements, launch instructions, and troubleshooting.

MAEG4060 Course Project

User Guide

Interactive Physical Simulation of Off-Road Vehicle Dynamics

Guo Tsun Hei; Yim Fu Chong

April 2026

1 How to start the application

- Open the `Game_Build` folder in your submission package (or run from the repository root after a local build; see below).
- **Double-click** `maeg4060_stage2.exe` in that folder to launch the program.
- Keep the executable beside the `Assets` folder so paths such as `Assets/models/model_jeep.x` resolve. If models fail to load, confirm the working directory contains `Assets`.

1.1 Building from source (optional)

The project is a CMake target (`maeg4060_stage2`) that needs legacy **D3DX** headers and import libraries (`d3dx9.h`, `d3dx9math.h`, etc.), which are *not* supplied by the Windows 10/11 SDK alone. CMake resolves them in one of two ways: (1) set `DXSDK_DIR` to the root of the **June 2010 DirectX SDK** (must contain `Include` and `Lib`); or (2) leave `DXSDK_DIR` unset and let CMake download the `Microsoft.DXSDK.D3DX` NuGet package into the build directory on first configure (requires network access; configure may take several minutes).

Recommended (matches `scripts/configure_build_release.bat`): run `vcvars64.bat` from your Visual Studio installation so `cl` is on `PATH`, then from the repository root use the Ninja generator and select MSVC explicitly, for example:

```
cmake -B build -G Ninja -DCMAKE_BUILD_TYPE=Release ^
  -DCMAKE_CXX_COMPILER=cl -DCMAKE_C_COMPILER=cl
cmake --build build
```

On some setups the Visual Studio CMake generator fails to detect a C++ compiler even after `vcvars64`; Ninja with `-DCMAKE_CXX_COMPILER=cl` avoids that. If you change the generator in an existing build tree, delete the `build` folder (or remove `CMakeCache.txt` and the `CMakeFiles` directory) before re-running `cmake`.

If the compiler stops with **C1060** (out of heap space), retry the build with less parallelism, e.g. `cmake -build build -j 1`, and close other heavy applications.

Alternatively, from the repository root: `cmake -B build -G "Visual Studio 17 2022" -A x64`, then `cmake -build build -config Release`, provided the toolset is detected correctly.

To package binaries: run `scripts/package_game_build.bat` to populate `Game_Build`, or `cmake -install build -prefix Game_Build`.

1.2 GitHub Actions (optional)

If `.github/workflows/windows-build.yml` is enabled on your fork, the **Windows build** workflow on `windows-latest` produces an artifact (zip) containing `maeg4060_stage2.exe`,

`D3DX9_43.dll` when the NuGet D3DX path is used, and a copy of `Assets/`. Download the artifact, unzip on Windows, and run the executable from that folder (keep `Assets` beside the `.exe`).

2 Hardware and software requirements

- **Platform:** Windows PC (64-bit Windows recommended).
- **Graphics API:** The application uses **DirectX 9.0c**. You need a DirectX 9-compatible GPU and the **DirectX 9 End-User Runtime** (or equivalent system components) installed as required by your Windows version.
- **Display:** A desktop resolution of at least **1280×720** is recommended; other resolutions are supported when the window is resized.
- **Optional input:** An **XInput-compatible gamepad** (e.g. Xbox controller) for analog control. Keyboard control works without a gamepad.

3 Controls

3.1 Keyboard (WASD and related keys)

Input is handled via the Win32 keyboard API (`GetAsyncKeyState`) with a vehicle-style layout, for example:

Input	Action
W / S	Forward / reverse (throttle)
A / D	Steer left / right
Space	Brake (or hand brake, depending on build)
M	Pause / resume windmill motion (Maya animation when present, or simple spin for the static mesh f
Esc	Quit application

Exact key bindings may match the on-screen HUD or readme in the binary package; the layout above reflects the project design (WASD movement with separate brake).

3.2 Gamepad (XInput)

With an XInput controller connected, the application maps analog axes and triggers for smoother control than the keyboard:

Control	Action
Left stick	Steering
Right trigger	Throttle (accelerate)
Left trigger	Brake / partial reverse

Triggers provide analog throttle similar to a real vehicle; keyboard W/S act as digital throttle inputs.

4 Troubleshooting

- **Missing DLLs or DirectX errors:** Install or repair the DirectX 9 end-user runtime and ensure GPU drivers are up to date.
- **Black screen or failed device creation:** Try windowed mode, update graphics drivers, and confirm no other application has exclusive full-screen access to the GPU.
- **Assets not found:** Run from `Game_Build` with the supplied `Assets` (or equivalent) folder beside the executable. For static prop replacements, place OBJ/MTL files under `Assets/models/replacements` (see `Assets/models/replacements/README.md`).
- **Gamepad not detected:** Use a wired connection or official wireless adapter; confirm the controller appears in Windows as an XInput device.